

Atividade de Segurança de Redes

Bernardo Rizzone 232013194

Fundamentos de Redes 2025/2

1. Criptografia Simétrica e Modos de Operação

a) Explique por que o modo ECB não deve ser utilizado em aplicações que envolvem padrões visuais ou dados estruturados. O modo ECB não é seguro para esse tipo

de dado porque ele cifra cada bloco de forma independente e idêntica. Se houver padrões repetidos no arquivo original (como o fundo de uma imagem ou campos repetidos num banco de dados), o ECB vai gerar blocos cifrados iguais. Isso faz com que a "estrutura" ou o desenho da imagem original continue visível mesmo depois de criptografada, vazando informação.

b) Compare os modos CBC e CTR quanto à paralelização, desempenho e vulnerabilidade a ataques.

- **Paralelização:** O CBC é sequencial (um bloco depende do anterior), então não dá pra paralelizar na cifragem. O CTR funciona como um contador independente, então é totalmente paralelizável (muito mais rápido).
- **Desempenho:** Por causa da paralelização, o CTR tende a ser mais rápido e permite acesso aleatório aos dados. O CBC é mais lento e exige leitura sequencial.
- **Vulnerabilidade:** O CBC sofre com ataques de Padding Oracle se não for bem implementado. O CTR é "maleável" (inverter um bit no cifrado inverte no original), então ele exige autenticação extra para ser seguro.

c) Dê um exemplo de aplicação real em que o modo CTR seria mais indicado que o CBC e justifique.

Serviços de streaming de vídeo (tipo Netflix ou YouTube). O CTR é ideal aqui porque permite "seek" (pular para o meio do vídeo) instantaneamente, já que não precisa decifrar tudo o que veio antes, e a alta velocidade ajuda a processar vídeos em 4K sem travar.

2. Criptografia Assimétrica e Troca de Chaves

a) Descreva como o protocolo TLS utiliza criptografia assimétrica apenas na etapa inicial da comunicação. O TLS usa criptografia híbrida para ganhar desempenho. A criptografia assimétrica é muito lenta e pesada

computacionalmente. Por isso, ela é usada apenas no handshake inicial para autenticar o servidor e trocar a chave de sessão de forma segura. Assim que a conexão é estabelecida, o protocolo muda para criptografia simétrica (como AES), que é muito mais rápida para transferir os dados da navegação.

b) Compare RSA, Diffie-Hellman e ECC em relação à segurança, desempenho e tamanho das chaves.

- **RSA:** Baseado em fatoração de números inteiros. Seguro, mas exige chaves muito grandes (2048+ bits) e é lento na geração de chaves.
- **Diffie-Hellman:** Focado especificamente na troca de chaves segura em canais públicos, não em assinatura.
- **ECC (Curvas Elípticas):** Baseado em logaritmo discreto em curvas. É o mais eficiente: com chaves pequenas (256 bits) oferece a mesma segurança de um RSA gigante, sendo mais rápido e consumindo menos energia.

c) Explique o impacto que o algoritmo de Shor teria sobre cada um desses sistemas. O algoritmo de Shor, rodando em um computador quântico com qubits suficientes, quebraria a base matemática de todos esses três (fatoração para RSA e logaritmo discreto para DH e ECC). Isso tornaria a criptografia assimétrica atual obsoleta, pois as chaves privadas poderiam ser descobertas rapidamente.

3. Funções de Hash e Assinaturas Digitais

a) Explique o papel da função hash na assinatura digital. A função hash garante eficiência. Em vez de criptografar o arquivo inteiro com a chave privada (o que seria muito lento), gera-se um "resumo" (hash) do arquivo. A

assinatura digital é aplicada apenas sobre esse hash. Isso garante a integridade (se o arquivo mudar, o hash muda) e autenticidade com muito mais velocidade.

b) Por que o uso de MD5 e SHA-1 é considerado inseguro atualmente? Porque foram encontradas vulnerabilidades de colisão nesses algoritmos. Isso significa que é possível gerar dois arquivos diferentes com o mesmo hash. Isso quebra a confiabilidade da assinatura digital, pois permitiria a falsificação de documentos.

c) Uma pequena alteração em um arquivo altera completamente o hash SHA-256. Nomeie e descreva a propriedade que explica esse comportamento. Essa propriedade é chamada de Efeito Avalanche. Ela garante que a alteração de um único bit na entrada resulte em uma alteração drástica (geralmente mudando cerca de 50% dos bits) no hash de saída, impedindo previsões ou engenharia reversa.

d) Cite uma aplicação prática onde a verificação de hash é essencial. Verificação de integridade de softwares baixados da internet (ISOs de sistemas operacionais, instaladores). O usuário compara o hash do arquivo baixado com o hash fornecido pelo desenvolvedor para garantir que o arquivo não foi corrompido ou alterado por malwares durante o download.

4. Infraestrutura de Chaves Públicas (PKI)

a) Descreva o papel da Autoridade Certificadora (CA) e da cadeia de confiança. A CA atua como uma entidade confiável ("cartório digital") que emite e assina certificados digitais, vinculando uma chave pública a uma identidade (site ou pessoa). A cadeia de confiança é a hierarquia de validação: o navegador confia na CA Raiz (pré-instalada no sistema), que assina CAs Intermediárias, que por sua vez assinam o certificado final do usuário. A validação ocorre seguindo esse caminho até encontrar uma Raiz confiável.

b) O que acontece quando uma CA raiz é comprometida? É um incidente catastrófico de segurança. Se a chave privada de uma CA Raiz for roubada, atacantes podem gerar certificados válidos para qualquer domínio (ex: google.com, bancos), permitindo ataques Man-in-the-Middle indetectáveis. A CA Raiz deve ser removida imediatamente das lojas de confiança de navegadores e sistemas operacionais, o que geralmente leva à falência da entidade operadora da CA.

c) Compare CRL e OCSP quanto ao funcionamento e à eficiência.

- CRL (Certificate Revocation List): A CA publica periodicamente uma lista de certificados inválidos. É menos eficiente pois o cliente precisa baixar a lista (que pode ser grande) e existe um tempo de atraso (delay) entre a revogação real e a publicação da nova lista.
- OCSP (Online Certificate Status Protocol): O cliente consulta o status de um certificado específico em tempo real diretamente nos servidores da CA. É mais eficiente e atualizado, mas gera latência na conexão e preocupações de privacidade.

d) Cite um exemplo real de falha ou comprometimento de uma CA e seus impactos. O caso da DigiNotar (2011). A CA holandesa foi comprometida e emitiu certificados falsos para domínios como google.com, usados para interceptar tráfego de usuários no Irã. Como consequência, navegadores como Chrome e Firefox revogaram a confiança na DigiNotar, levando a empresa à falência e dissolução.

5. Criptografia Pós-Quântica

a) Explique a diferença do impacto da computação quântica em algoritmos simétricos e assimétricos. A computação quântica afeta os dois de forma desigual.

- Assimétricos (RSA, ECC): Serão completamente quebrados pelo Algoritmo de Shor, que resolve fatoração e logaritmo discreto em tempo polinomial.
- Simétricos (AES, Hash): Serão enfraquecidos pelo Algoritmo de Grover, que reduz a segurança efetiva pela metade (raiz quadrada do espaço de chaves). Eles não são "quebrados", apenas precisam de chaves maiores.

b) Quais algoritmos estão sendo padronizados pelo NIST para resistir a ataques quânticos? O NIST selecionou recentemente o algoritmo CRYSTALS-Kyber para

encapsulamento de chaves (KEM) e cifragem. Para assinaturas digitais, os principais selecionados foram CRYSTALS-Dilithium, FALCON e SPHINCS+.

c) **Por que AES-256 e SHA-3 ainda são considerados seguros mesmo diante de computadores**

quânticos? Devido à natureza do Algoritmo de Grover. Como ele reduz a complexidade de força bruta pela raiz quadrada, o AES-256 teria uma segurança efetiva de 128 bits (ainda inquebrável atualmente). O mesmo vale para o SHA-3 com saídas grandes (ex: 384 ou 512 bits); mesmo com a redução quadrática na dificuldade de encontrar colisões, o esforço computacional necessário permaneceria inviável.

6. Labortatorio 1- Cifragem Avançada com AES

```
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ openssl rand -hex 32 > chave.key
openssl rand -hex 16 > iv.key
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ openssl enc -aes-256-ecb -in imagem.bmp -out img_ecb.bmp
-K $(cat chave.key)
openssl enc -aes-256-cbc -in imagem.bmp -out img_cbc.bmp -K $(cat chave.key) -iv $(cat iv.key)
openssl enc -aes-256-ctr -in imagem.bmp -out img_ctr.bmp -K $(cat chave.key) -iv $(cat iv.key)
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ dd if=imagem.bmp of=img_ecb.bmp bs=54 count=1 conv=notru
nc
dd if=imagem.bmp of=img_cbc.bmp bs=54 count=1 conv=notrunc
dd if=imagem.bmp of=img_ctr.bmp bs=54 count=1 conv=notrunc
1+0 records in
1+0 records out
54 bytes copied, 0.000988733 s, 54.6 kB/s
1+0 records in
1+0 records out
54 bytes copied, 0.000905831 s, 59.6 kB/s
1+0 records in
1+0 records out
54 bytes copied, 0.00103513 s, 52.2 kB/s
```

Imagem ECB

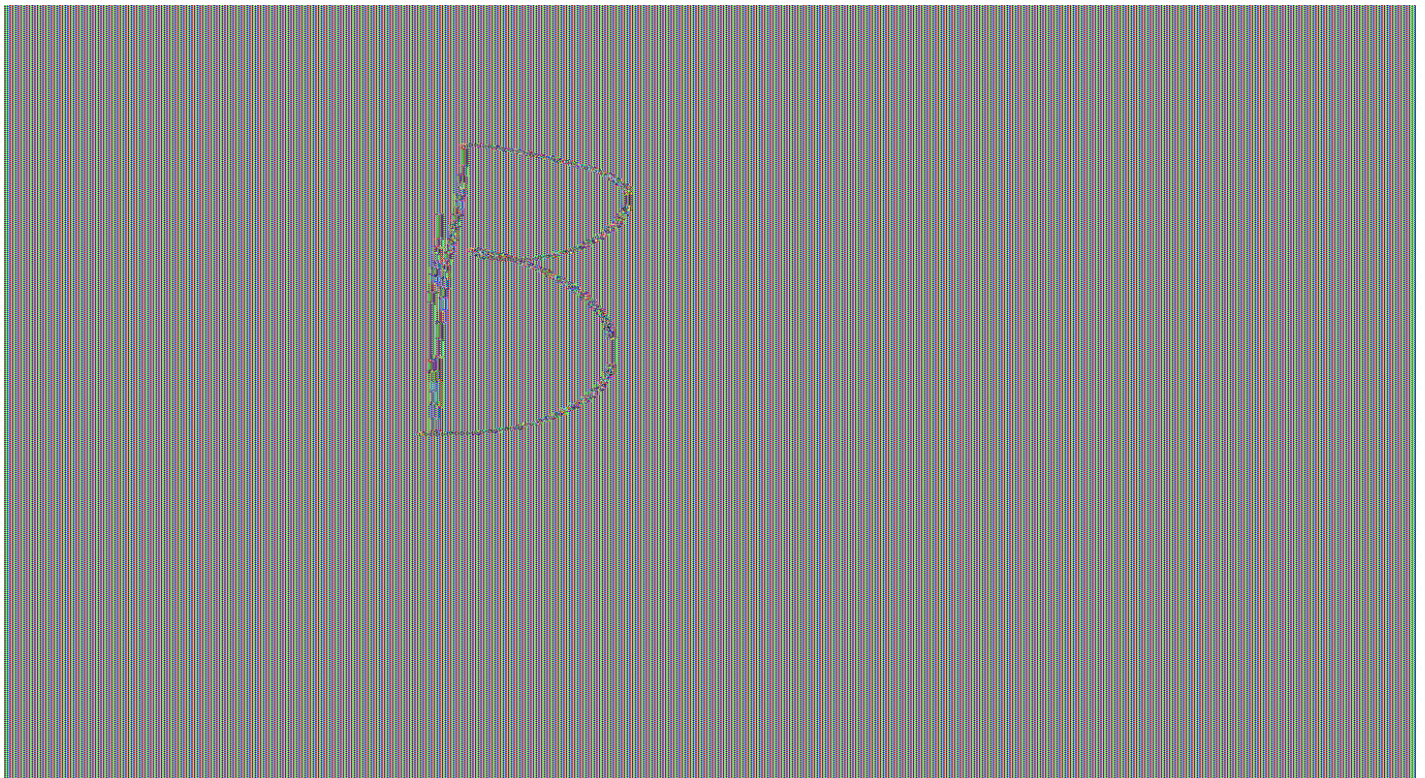


Imagem CBC

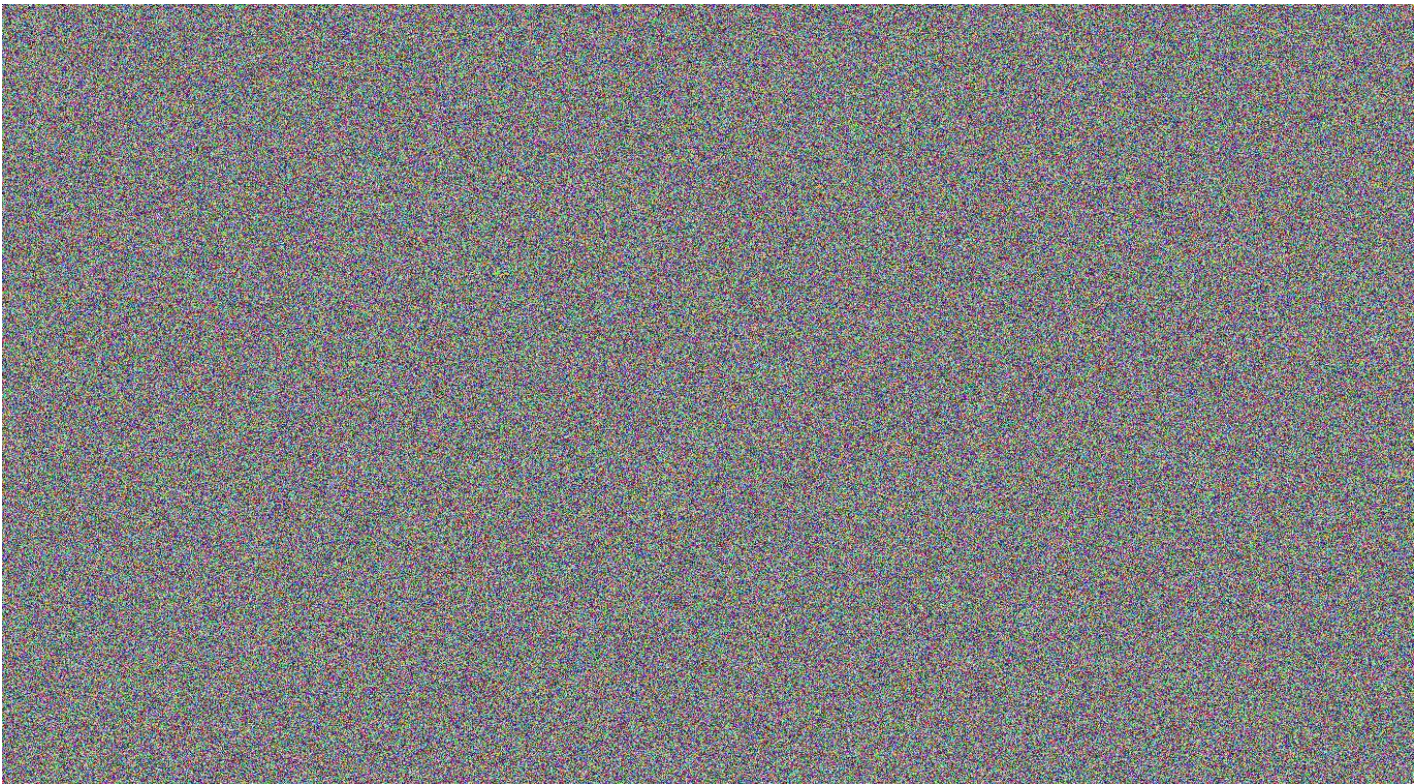
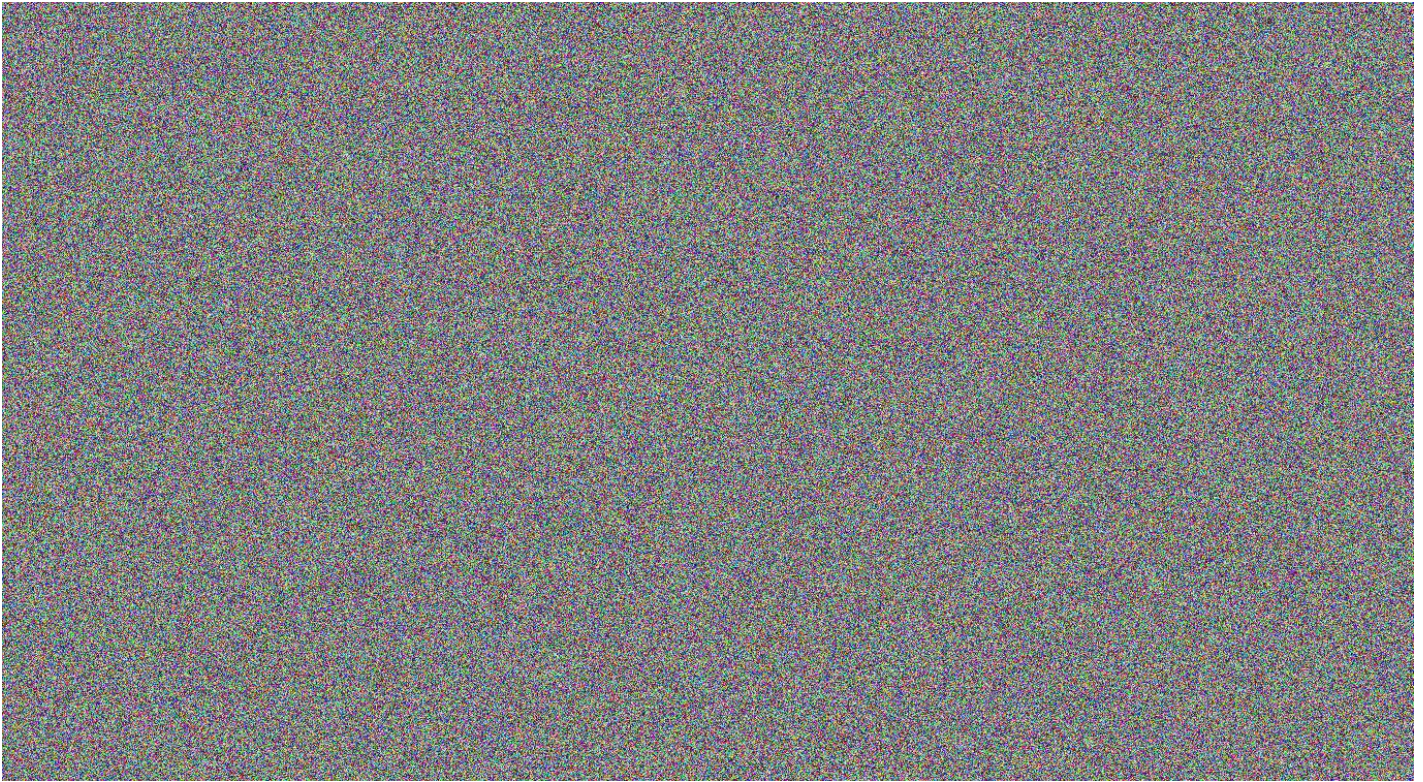


Imagem CTR



- **Qual apresenta padrões visuais?** É o ECB. Como ele cifra cada pedacinho igual da mesma forma (ex: um pedaço de fundo branco vira sempre o mesmo código cifrado), ele preserva a "forma" da imagem.
- **Por que o IV é essencial?** Ele é o "tempero" aleatório. Sem o IV, se eu mandar a mensagem "Oi" duas vezes hoje com a mesma chave, o código cifrado sai igual. O IV garante que, mesmo repetindo a mensagem e a chave, o resultado final seja sempre diferente e imprevisível.
- **Qual é totalmente paralelo?** É o CTR. O processador não precisa esperar o começo da imagem ser cifrado para cifrar o final. Ele pode fazer tudo ao mesmo tempo porque cada bloco é independente.


```
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:BR
State or Province Name (full name) [Some-State]:DF
Locality Name (eg, city) []:Brasilia
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:Minha CA Raiz
Email Address []:

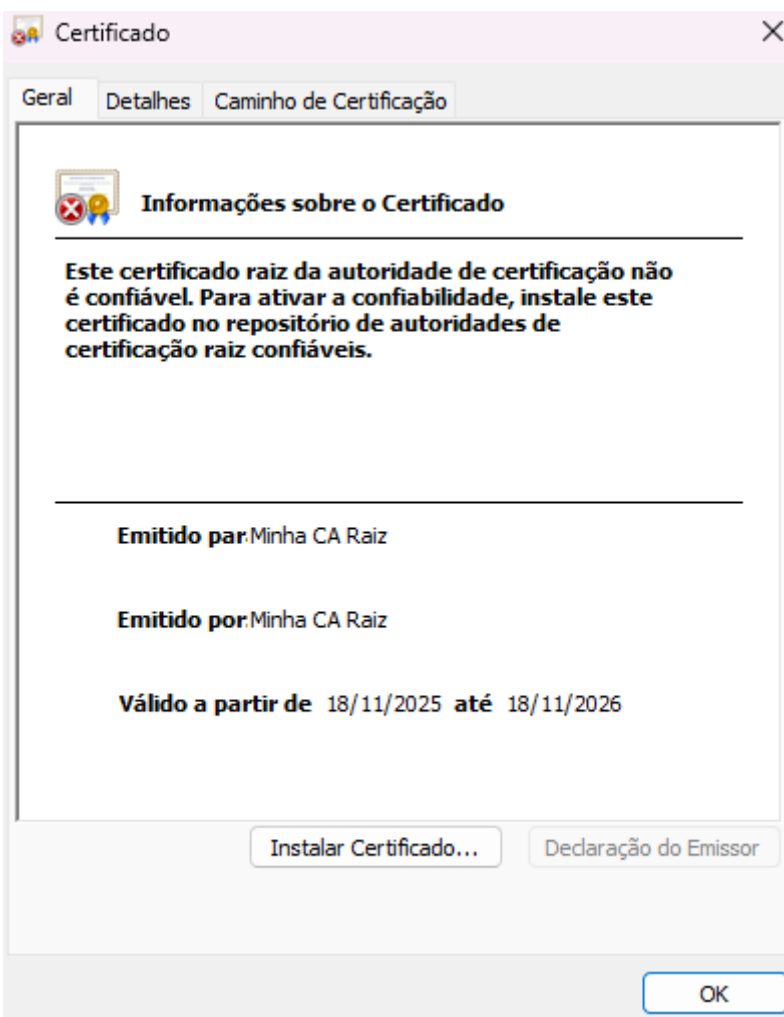
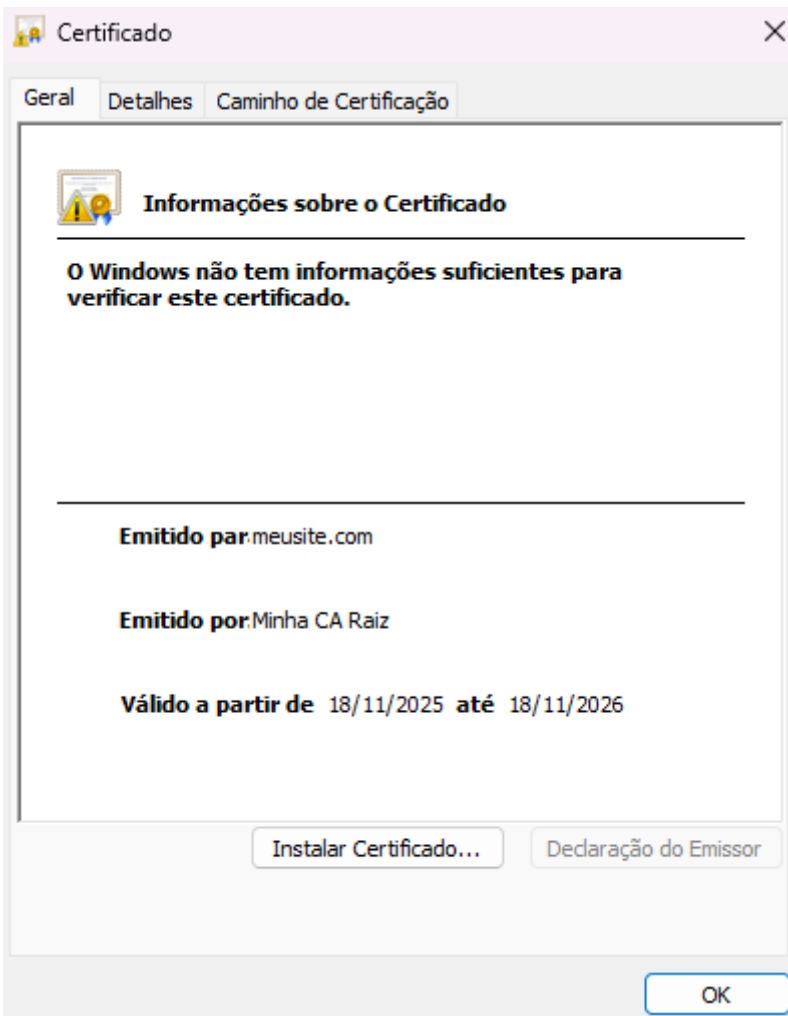
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ openssl genrsa -out servidor.key 2048
openssl req -new -key servidor.key -out servidor.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:BR
State or Province Name (full name) [Some-State]:DF
Locality Name (eg, city) []:Brasilia
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:meusite.com
Email Address []:

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:fofodom
An optional company name []:
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ openssl x509 -req -in servidor.csr -CA ca.crt -CAkey ca.
key -CAcreateserial -out servidor.crt -days 365 -sha256
Certificate request self-signature ok
subject=C = BR, ST = DF, L = Brasilia, O = Internet Widgits Pty Ltd, CN = meusite.com
```

```
rizzonly@DESKTOP-G55DH5D:/mnt/c/Users/User/Desktop$ openssl x509 -in servidor.crt -text -noout
Certificate:
    Data:
        Version: 1 (0x0)
        Serial Number:
            39:42:36:90:ea:42:cf:88:00:2e:1e:66:3d:1f:51:7a:98:f4:08:19
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = BR, ST = DF, L = Brasilia, O = Internet Widgits Pty Ltd, CN = Minha CA Raiz
        Validity
            Not Before: Nov 18 19:37:10 2025 GMT
            Not After : Nov 18 19:37:10 2026 GMT
        Subject: C = BR, ST = DF, L = Brasilia, O = Internet Widgits Pty Ltd, CN = meusite.com
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:d7:20:67:57:0e:bd:80:32:f1:7d:7b:cb:75:e6:
                46:ab:26:a3:dd:31:05:46:af:36:d2:cd:5b:b2:c1:
                05:ee:1f:35:82:a1:77:af:74:b9:11:19:97:e5:c6:
                43:07:fd:a3:e9:5f:fd:62:bd:54:f9:71:be:5a:01:
                26:4c:ce:6f:a8:15:02:ef:b1:df:11:82:7b:16:d5:
                fa:12:86:08:10:ca:16:1f:df:68:a0:d1:a9:ce:e3:
                b3:d4:b5:b3:a6:11:d3:66:3d:c9:a3:7b:7b:85:cb:
                6c:c5:65:8d:c4:ba:95:11:42:f2:c4:71:a1:fd:26:
                28:3b:90:88:c0:e7:c2:08:05:b1:86:df:d6:15:7b:
                c7:8f:3a:ea:4d:18:fa:91:10:47:fa:e3:74:c2:24:
                50:9d:0b:99:7b:56:b2:76:52:e3:97:e5:b2:5b:70:
                0d:42:14:80:c1:77:d2:68:22:6c:cf:8b:b4:da:d7:
```

```
                Exponent: 65537 (0x10001)
        Signature Algorithm: sha256WithRSAEncryption
        Signature Value:
            81:5a:57:92:a5:6c:6d:bc:df:35:2f:6a:f2:ea:a7:2d:f0:9a:
            dd:4d:d1:85:14:e3:5d:de:b6:25:e1:e8:a2:44:15:37:2e:03:
            fa:3e:ed:86:f4:6e:07:81:66:c4:8f:0b:b7:35:24:3e:e1:23:
            30:a6:ce:a5:c5:06:43:9c:39:0e:55:8a:26:d2:b4:e4:22:4c:
            7e:2c:30:68:82:2d:d2:37:2d:a9:70:01:ec:cb:35:94:7f:c4:
            bc:01:50:00:30:60:b0:8b:57:6a:95:63:ff:04:b1:9e:de:cf:
            83:ff:9d:0b:6a:27:a8:5b:70:44:b1:e9:67:bc:a4:24:9c:61:
            fa:fc:1a:01:49:1e:d7:0f:a9:ad:3b:8f:2c:7d:fa:8b:ee:99:
            f1:86:da:cc:4e:3d:38:d9:43:17:1a:e0:cb:0e:72:a7:ff:00:
            6e:63:b8:c3:e0:77:5a:30:07:29:3c:e6:85:d9:11:37:3d:ad:
            81:f8:49:6b:cb:62:c1:aa:c0:63:a0:35:1e:f3:36:81:f9:41:
            e8:dc:34:a0:16:83:64:4c:1b:b6:81:70:0b:3c:f7:d8:be:88:
            71:d1:6d:b8:e5:74:3a:f9:ee:f8:34:fc:31:d0:5b:bc:e2:bb:
            0f:ba:36:e4:cf:e0:07:44:c6:ec:10:69:55:44:01:31:65:3f:
            a3:c7:d8:17:83:d5:3d:73:92:1e:32:fb:6b:bc:e7:86:0b:28:
            e4:66:75:92:f8:00:1a:2a:37:3f:29:27:ff:6d:51:68:17:09:
            4f:93:3f:d0:21:0f:b0:08:05:3b:f3:15:09:ac:72:78:18:45:
            ba:3b:06:74:24:4d:20:0d:30:ce:52:07:61:2f:dd:c0:3a:8a:
            e4:fe:7a:27:a9:41:1c:08:c2:f9:7b:93:95:c7:53:fe:07:74:
            86:cb:5f:b9:c5:7e:40:1e:16:0e:39:b8:50:a8:44:b9:5a:c4:
            7a:72:b3:fc:fd:ff:27:a5:0b:bd:ad:d5:18:01:da:8c:8f:6d:
            3f:41:6a:e7:f1:24:71:ab:01:6d:c4:50:8a:da:ce:f8:a9:60:
            99:0a:c3:d0:92:ab:89:3b:18:60:6d:8c:0d:82:4f:f0:93:b1:
            ff:78:b9:5d:25:c2:a1:a7:12:ee:c6:cd:c4:d7:bd:83:ae:10:
            c4:41:5d:13:9c:f1:9b:05:60:11:1b:f2:1e:3b:c2:31:2f:00:
            08:bb:47:aa:79:d2:9e:f0:37:2e:20:95:c7:d2:46:d2:10:82:
            f6:a4:16:f5:7e:54:38:b1:5b:f6:05:1b:25:83:16:c0:8f:5a:
```

Certificados:



- **Subject (Sujeito):** É o "dono" do certificado. No seu caso, vai aparecer o meusite.com (ou o que você digitou no Common Name).
- **Issuer (Emissor):** É quem assinou e garantiu que o sujeito é confiável. Vai aparecer o nome da sua CA ("Minha CA Raiz").
- **Cadeia de Confiança:** Quando você entra num site, o navegador olha o certificado dele. Vê que foi assinado pela "CA X". O navegador verifica se confia na "CA X". Se sim, cadeado verde. Se não, tela vermelha de erro.
- **A Cadeia de Confiança (O "Quem indicou?") Quando você entra num site HTTPS:**
 - O site te manda o certificado dele.
 - Seu navegador olha pro Issuer e pergunta: "Eu conheço esse cara?".
 - Se não conhecer direto, ele procura o certificado de quem assinou o Issuer (CAs Intermediárias).
 - Ele vai subindo essa escada até chegar na CA Raiz. A CA Raiz já vem instalada de fábrica no Windows/Linux/Android.
 - Se a "corrente" ligar o site até essa Raiz que já tá no seu PC, o cadeado fica verde. Se a corrente quebrar no meio (faltar um certificado intermediário) ou a Raiz não for confiável, dá tela vermelha.

Se a CA for comprometida. Se um hacker roubar a chave privada da CA (o ca.key que você criou), ele vira "Deus". Ele pode criar um certificado para o facebook.com, assinar com essa chave roubada, e o seu navegador vai achar que é legítimo! Isso permite ataques Man-in-the-Middle perfeitos. Você entraria no site do banco, veria o cadeado verde, mas estaria falando com o hacker. A única solução é a CA avisar o mundo todo (Google, Microsoft, Apple) para pararem de confiar nela imediatamente.